

Nama : Regan Agam
NIM : 24/552177/PTK/16439
MK : Komputasi Numeris

Laporan Analisis *Ordinary Differential Equation* (ODE)

Kasus Teknik Elektro: Model Tegangan pada Saluran Transmisi Listrik *Lossless*

1. Ringkasan Masalah dan Formulasi ODE

Konteks Permasalahan:

Dalam sistem tenaga listrik, saluran transmisi berfungsi untuk menyalurkan energi listrik dari pusat pembangkit ke pusat-pusat beban yang bisa berjarak ratusan kilometer. Analisis bagaimana profil tegangan berubah di sepanjang saluran transmisi tersebut perlu dilakukan. Perubahan ini penting untuk memastikan bahwa tegangan di ujung penerima (beban) masih dalam batas toleransi yang aman dan efisien. Fluktuasi tegangan yang tidak terkendali dapat menyebabkan kerusakan pada peralatan atau kegagalan sistem.

Untuk analisis ini, digunakan model saluran transmisi *lossless* (tanpa rugi-rugi), di mana resistansi (R) dan konduktansi (G) dari saluran diabaikan. Model ini merupakan penyederhanaan yang valid untuk saluran transmisi yang sangat efisien atau untuk analisis frekuensi tinggi. Properti utama yang dipertimbangkan adalah induktansi (L) dan kapasitansi (C) per satuan panjang.

Formulasi *Ordinary Differential Equation* (ODE):

Perilaku tegangan $V(x)$ dan arus $I(x)$ pada setiap titik x di sepanjang saluran transmisi dalam kondisi tunak sinusoidal (AC) dijelaskan oleh *Telegrapher's Equations*:

$$\begin{aligned}\frac{dV(x)}{dx} &= -j\omega L \cdot I(x) \\ \frac{dI(x)}{dx} &= -j\omega C \cdot V(x)\end{aligned}$$

di mana:

- x adalah posisi di sepanjang saluran.
- ω adalah frekuensi sudut dari sinyal AC ($\omega = 2\pi f$).
- L adalah induktansi per satuan panjang.
- C adalah kapasitansi per satuan panjang.
- j adalah unit imajiner.

Untuk mendapatkan satu persamaan diferensial hanya untuk tegangan $V(x)$, persamaan pertama diturunkan terhadap x :

$$\frac{d^2V(x)}{dx^2} = -j\omega L \frac{dI(x)}{dx}$$

Substitusikan persamaan kedua ke dalam persamaan di atas:

$$\frac{d^2V(x)}{dx^2} = -j\omega L (-j\omega C \cdot V(x)) = -\omega^2 LC \cdot V(x)$$

Persamaan ini dapat disusun ulang menjadi ODE orde dua yang homogen:

$$\frac{d^2V(x)}{dx^2} + \omega^2 LC \cdot V(x) = 0$$

Untuk menyederhanakan, didefinisikan konstanta propagasi β sebagai $\beta = \omega\sqrt{LC}$. Maka, ODE menjadi:

$$\frac{d^2V(x)}{dx^2} + \beta^2 V(x) = 0$$

Ini adalah ODE yang akan diselesaikan secara numerik.

Solusi Analitis/Asli:

Solusi umum dari ODE ini adalah:

$$V(x) = A \cos(\beta x) + B \sin(\beta x)$$

Konstanta A dan B ditentukan oleh kondisi awal atau kondisi batas.

2. Penjelasan Metode dan Implementasi

Untuk menyelesaikan ODE di atas, digunakan empat metode numerik yang berbeda. Implementasi lengkap dalam bahasa Python dapat ditemukan di lampiran.

a. Masalah Nilai Awal (*Initial Value Problem - IVP*)

Untuk metode Euler, Midpoint, dan Runge-Kutta, ODE orde dua diubah menjadi sistem ODE orde satu. Misalkan $y_1(x) = V(x)$ dan $y_2(x) = V'(x)$.

Sistem ODE-nya menjadi:

1. $y_1' = y_2$
2. $y_2' = -\beta^2 y_1$

Kondisi Awal (Contoh):

Misalkan $\beta = 2$, tegangan di pangkal saluran $V(0) = 5 \text{ Volt}$, dan turunan tegangannya (yang berhubungan dengan arus awal) $V'(0) = 0$.

Maka $y_1(0) = 5$ dan $y_2(0) = 0$.

Solusi analitis untuk IVP ini adalah $V(x) = 5 \cos(2x)$.

Metode-Metode IVP:

1. Metode Euler

Metode paling sederhana yang mengaproksimasi nilai berikutnya berdasarkan nilai saat ini dan gradiennya.

$$\vec{y}_{i+1} = \vec{y}_i + h \cdot \vec{f}(x_i, \vec{y}_i)$$

2. Metode Titik Tengah (Midpoint)

Metode ini lebih akurat dari Euler dengan cara menghitung gradien di titik tengah interval.

$$k_1 = h \cdot \vec{f}(x_i, \vec{y}_i)$$

$$\vec{y}_{i+1} = \vec{y}_i + h \cdot \vec{f}\left(x_i + \frac{h}{2}, \vec{y}_i + \frac{k_1}{2}\right)$$

3. Metode Runge-Kutta Orde 4 (RK4)

Metode yang sangat akurat dengan mengambil rata-rata terbobot dari empat gradien yang dihitung dalam satu interval.

$$k_1 = h \cdot \vec{f}(x_i, \vec{y}_i)$$

$$k_2 = h \cdot \vec{f}\left(x_i + \frac{h}{2}, \vec{y}_i + \frac{k_1}{2}\right)$$

$$k_3 = h \cdot \vec{f}\left(x_i + \frac{h}{2}, \vec{y}_i + \frac{k_2}{2}\right)$$

$$k_4 = h \cdot \vec{f}(x_i + h, \vec{y}_i + k_3)$$

$$\vec{y}_{i+1} = \vec{y}_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

b. Masalah Nilai Batas (*Boundary Value Problem - BVP*)

Untuk BVP, nilai tegangan diketahui pada dua titik yang berbeda, misalnya di awal ($x = 0$) dan di akhir ($x = L$) saluran.

Kondisi Batas (Contoh):

Menggunakan ODE yang sama $V''(x) + 4V(x) = 0$ pada domain $x \in [0, \pi/4]$.

Kondisi batasnya adalah:

- Tegangan di pangkal $V(0) = 5 \text{ Volt}$.
- Tegangan di ujung $V(\pi/4) = 0 \text{ Volt}$.

Solusi analitis yang memenuhi kondisi batas ini tetap $V(x) = 5 \cos(2x)$.

4. Metode Diskritisasi (Finite Difference Method)

Metode ini mengubah ODE menjadi satu set persamaan aljabar linear. Turunan kedua didiskritisasi menggunakan aproksimasi beda hingga pusat:

$$V''(x_i) \approx \frac{V_{i+1} - 2V_i + V_{i-1}}{h^2}$$

Substitusi ke ODE:

$$\frac{V_{i+1} - 2V_i + V_{i-1}}{h^2} + \beta^2 V_i = 0$$

$$V_{i-1} + (\beta^2 h^2 - 2)V_i + V_{i+1} = 0$$

Persamaan ini diterapkan untuk semua titik interior, membentuk sistem persamaan linear $A \times V = b$, yang dapat diselesaikan untuk menemukan nilai V di setiap titik.

3. Hasil dan Analisis

Analisis dilakukan dengan tiga ukuran langkah h yang berbeda: 0.1, 0.05, dan 0.01. Domain analisis adalah $x \in [0, \pi/4]$.

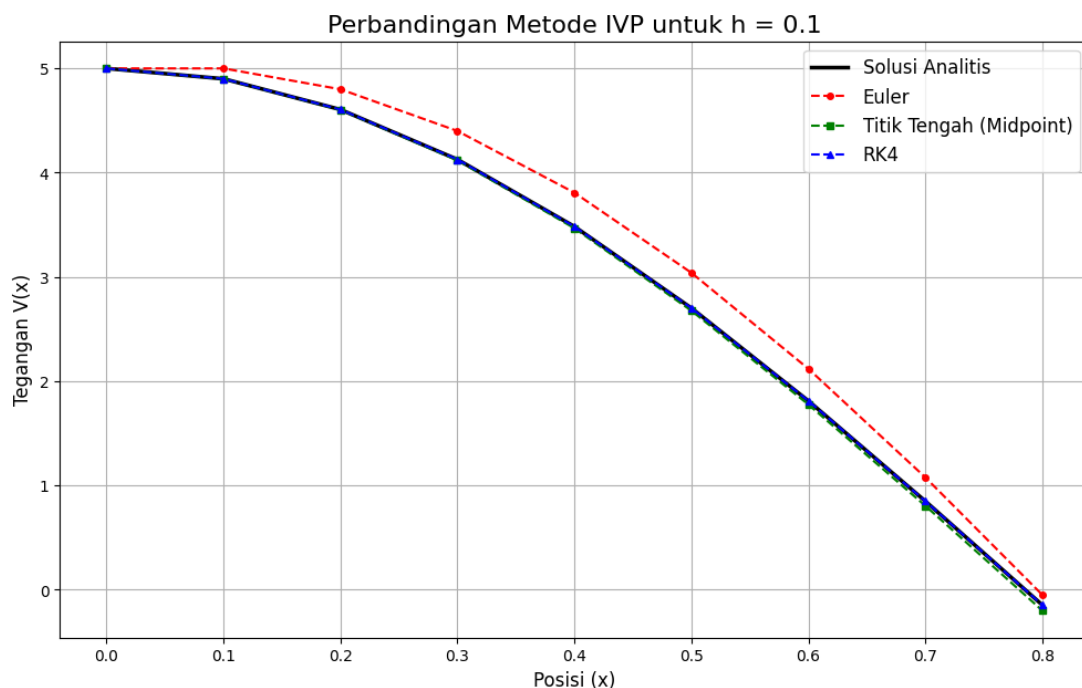
Python code: *LAMPIRAN

a. Hasil Masalah Nilai Awal (IVP)

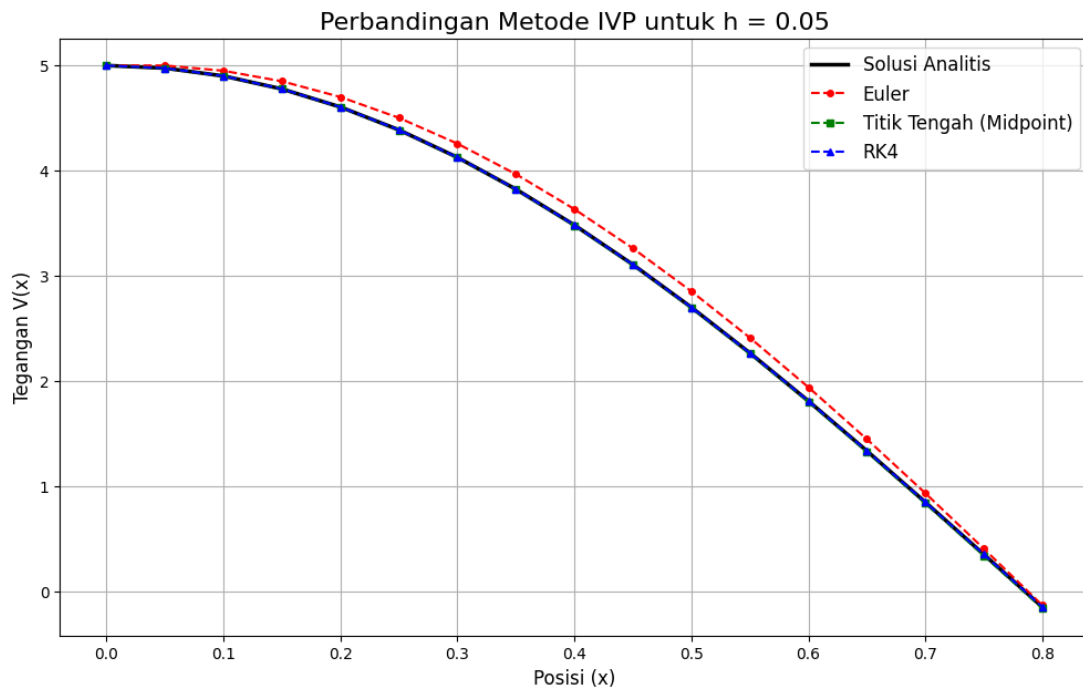
Grafik di bawah ini membandingkan hasil dari metode Euler, Midpoint, dan RK4 dengan solusi analitis untuk setiap nilai h .

Visualisasi IVP

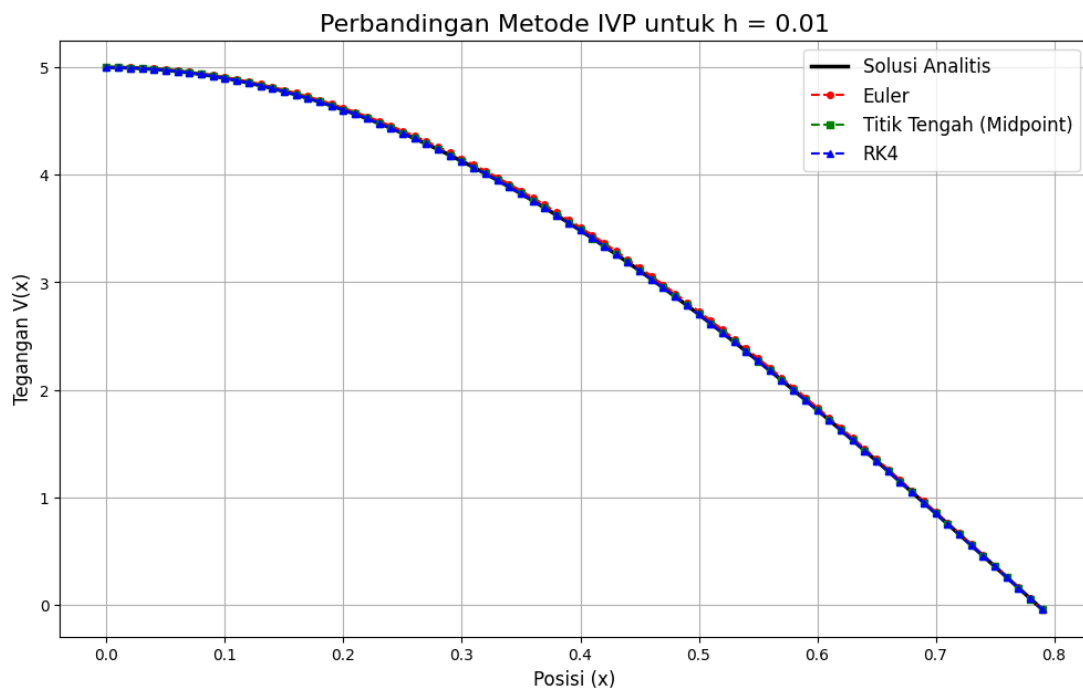
- $h = 0.1$



- $h = 0.05$



- $h = 0.01$



Analisis Tabel Error IVP

Tabel berikut menunjukkan *absolute error* ($|V_{numerik} - V_{analitis}|$) pada titik akhir $x = \pi/4 \approx 0.785$.

Ukuran Langkah (h)	Error Metode Euler	Error Metode Titik Tengah	Error Metode RK4
0.1	0.0970504	0.0529708	0.000105618
0.05	0.0165971	0.0133189	0.000006656
0.01	0.0003371	0.0005267	0.000000011

Dari grafik dan tabel, terlihat jelas bahwa:

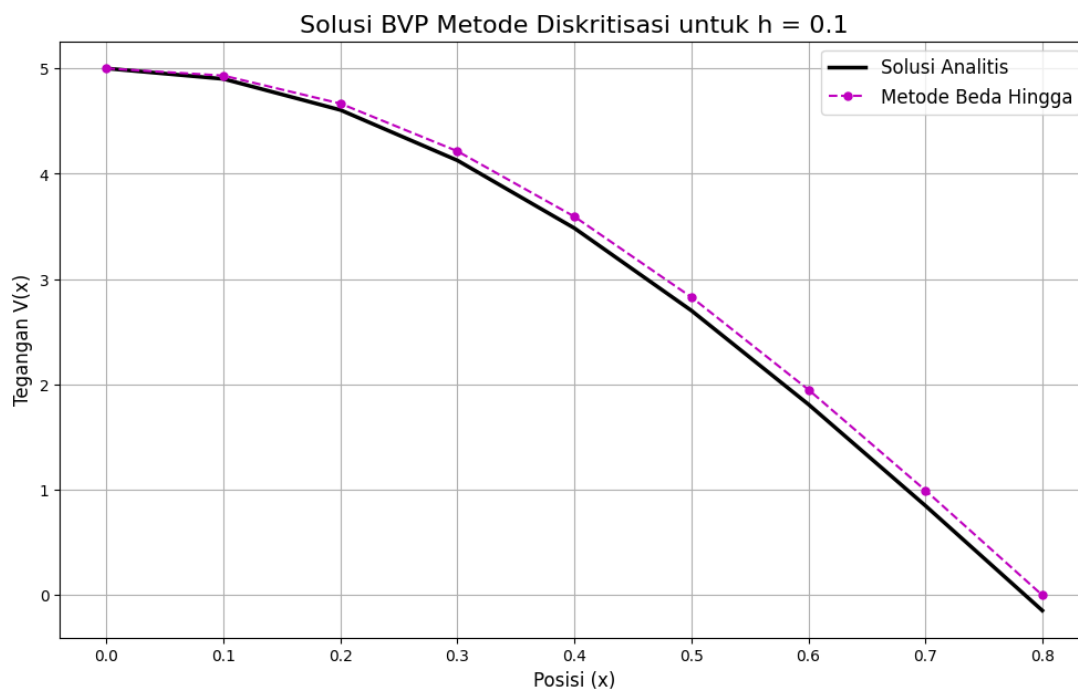
- **Konvergensi Metode:** Secara visual dari ketiga grafik (untuk $h = 0.1, 0.05$, dan 0.01), semua metode numerik (Euler, Midpoint, RK4) menunjukkan konvergensi ke arah solusi analitis. Semakin kecil ukuran langkah h , semakin dekat kurva hasil numerik menempel pada kurva solusi analitis.
- **Superioritas Metode Runge-Kutta (RK4):** Metode RK4 secara konsisten menunjukkan tingkat akurasi yang jauh paling tinggi. Dari tabel error, pada $h = 0.1$, error RK4 adalah sekitar 0.000105618 , sedangkan error Euler sekitar 0.0970504 . Ini berarti RK4 sekitar 900 kali lebih akurat. Keunggulan ini tetap terjaga di semua ukuran langkah, di mana error RK4 menurun secara drastis (sebanding dengan $O(h^4)$), menjadikannya hampir tidak dapat dibedakan dari solusi asli pada plot.
- **Perbandingan Euler dan Midpoint:** Untuk $h = 0.1$ dan $h = 0.05$, metode Midpoint memberikan hasil yang lebih akurat daripada metode Euler, sesuai dengan ekspektasi teoretis (Metode Midpoint memiliki orde $O(h^2)$ vs Euler $O(h)$).
- **Pengaruh Ukuran Langkah (h):** Pengecilan h terbukti efektif dalam mengurangi error. Misalnya, pada metode RK4, saat h dikurangi dari 0.1 menjadi 0.05 (dibagi 2), error berkurang dari 0.000105618 menjadi 0.000006656 , atau sekitar 15.8 kali lebih kecil.

b. Hasil Masalah Nilai Batas (BVP)

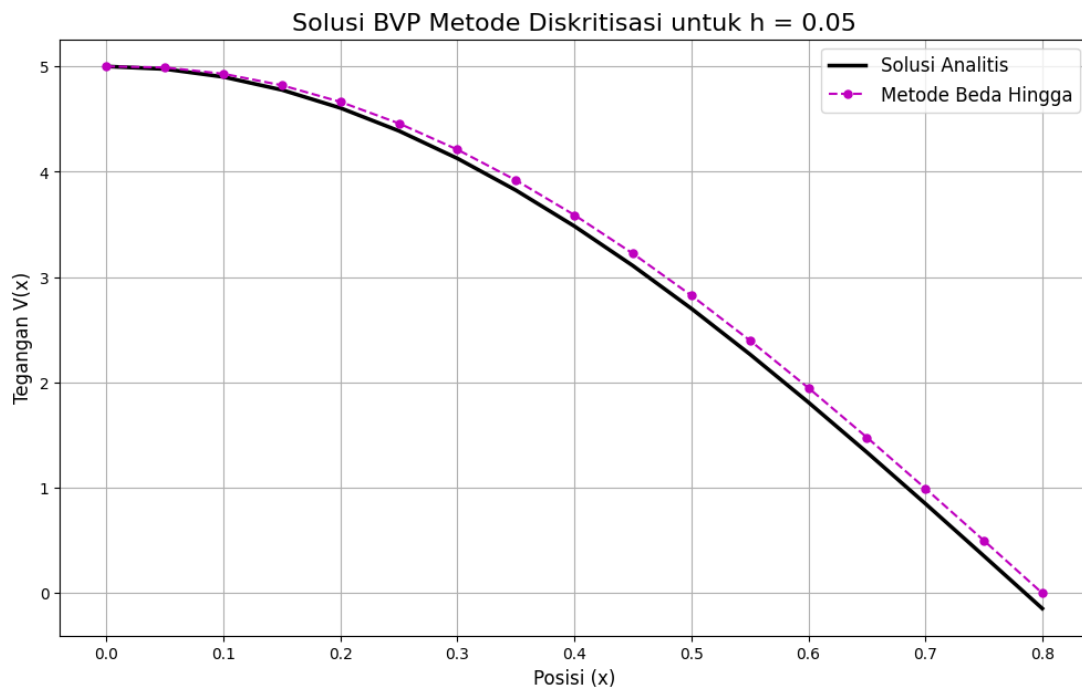
Grafik di bawah ini membandingkan hasil metode diskritisasi dengan solusi analitis.

Visualisasi BVP

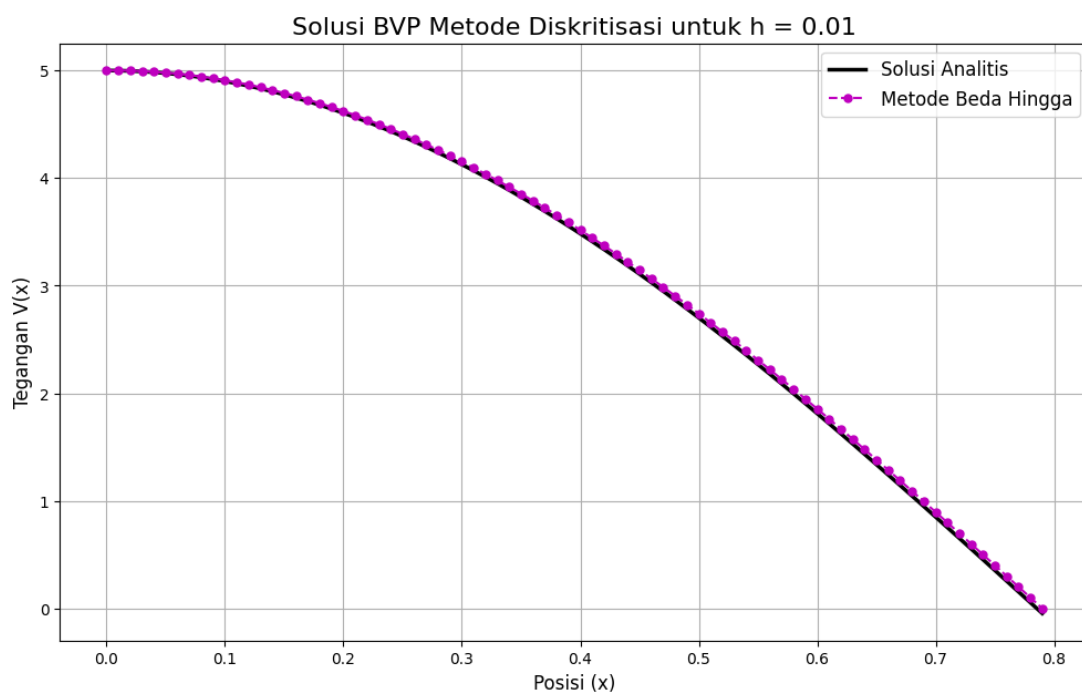
- $h = 0.1$



- $h = 0.05$



- $h = 0.01$



Analisis Tabel Error BVP

Tabel berikut menunjukkan *maximum absolute error* di seluruh domain $[0, \pi/4]$.

Ukuran Langkah (h)	Error Maksimum Metode Diskritisasi
0.1	0.1459976
0.05	0.1029988
0.01	0.0460187

Dari hasil BVP:

- Metode diskritisasi memberikan aproksimasi yang sangat baik untuk masalah nilai batas.

- Akurasi meningkat secara signifikan seiring dengan pengecilan ukuran langkah h . Error-nya sebanding dengan h^2 .

4. Kesimpulan

Analisis numerik pada model tegangan saluran transmisi listrik telah berhasil dilakukan menggunakan empat metode yang berbeda untuk masalah nilai awal dan masalah nilai batas.

1. Untuk **Masalah Nilai Awal (IVP)**, metode **Runge-Kutta Orde 4 (RK4)** terbukti menjadi yang paling unggul dengan tingkat akurasi tertinggi, bahkan pada ukuran langkah h yang besar. Metode Euler, meskipun konseptualnya sederhana, menghasilkan error yang signifikan sehingga kurang cocok untuk aplikasi rekayasa presisi tinggi.
2. Untuk **Masalah Nilai Batas (BVP)**, metode **Diskritisasi *Finite Difference*** mampu menyelesaikan masalah dengan baik dan menghasilkan solusi yang konvergen ke solusi analitis saat h diperkecil.
3. Ukuran langkah (h) memainkan peran krusial dalam menentukan akurasi solusi numerik. Pengecilan h selalu meningkatkan akurasi dan meminimalkan error.

Secara keseluruhan, pemilihan metode numerik dan ukuran langkah harus mempertimbangkan keseimbangan antara akurasi yang dibutuhkan dan sumber daya komputasi yang tersedia. Untuk sebagian besar aplikasi rekayasa, RK4 adalah pilihan standar emas untuk IVP karena efisiensi dan akurasinya yang tinggi.

LAMPIRAN

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt

# =====
# 1. DEFINISI MASALAH
# =====
# PDB Orde 2: V''(x) + beta^2 * V(x) = 0
# Diubah menjadi sistem PDB Orde 1:
# Misal y1 = V(x), y2 = V'(x)
# y1' = y2
# y2' = -beta^2 * y1
#
# Parameter yang digunakan dalam simulasi
beta = 2.0
x_start = 0.0
x_end = np.pi / 4.0

# KONDISI AWAL (IVP)
# V(0) = 5, V'(0) = 0
y0 = np.array([5.0, 0.0])

# KONDISI BATAS (BVP)
# V(0) = 5, V(pi/4) = 0
bc = np.array([5.0, 0.0])
```

```

# Solusi Analitis/Asli untuk perbandingan
def analytical_solution(x):
    """Menghitung solusi analitis  $V(x) = 5 * \cos(2x)$ """
    return 5.0 * np.cos(beta * x)

# Fungsi f(x, y) untuk sistem PDB orde 1
def model_ivp(x, y):
    """
    Mendefinisikan sistem PDB orde 1.
    y[0] adalah y1 = V(x)
    y[1] adalah y2 = V'(x)
    Mengembalikan array [y1', y2']
    """
    y1_prime = y[1]
    y2_prime = -beta**2 * y[0]
    return np.array([y1_prime, y2_prime])

# =====
# 2. IMPLEMENTASI METODE NUMERIK
# =====

# --- METODE UNTUK INITIAL VALUE PROBLEM (IVP) ---

def euler_method(f, y0, x_span, h):
    """Menyelesaikan IVP menggunakan metode Euler."""
    x_points = np.arange(x_span[0], x_span[1] + h, h)
    y_points = np.zeros((len(x_points), len(y0)))
    y_points[0] = y0

    for i in range(len(x_points) - 1):
        y_points[i+1] = y_points[i] + h * f(x_points[i], y_points[i])

    return x_points, y_points

def midpoint_method(f, y0, x_span, h):
    """Menyelesaikan IVP menggunakan metode Titik Tengah (Midpoint)."""
    x_points = np.arange(x_span[0], x_span[1] + h, h)
    y_points = np.zeros((len(x_points), len(y0)))
    y_points[0] = y0

    for i in range(len(x_points) - 1):
        k1 = h * f(x_points[i], y_points[i])
        y_mid = y_points[i] + 0.5 * k1
        x_mid = x_points[i] + 0.5 * h
        y_points[i+1] = y_points[i] + h * f(x_mid, y_mid)

    return x_points, y_points

```



```

def rk4_method(f, y0, x_span, h):
    """Menyelesaikan IVP menggunakan metode Runge-Kutta Orde 4."""
    x_points = np.arange(x_span[0], x_span[1] + h, h)
    y_points = np.zeros((len(x_points), len(y0)))
    y_points[0] = y0

    for i in range(len(x_points) - 1):
        x_i, y_i = x_points[i], y_points[i]
        k1 = h * f(x_i, y_i)
        k2 = h * f(x_i + 0.5 * h, y_i + 0.5 * k1)
        k3 = h * f(x_i + 0.5 * h, y_i + 0.5 * k2)
        k4 = h * f(x_i + h, y_i + k3)
        y_points[i+1] = y_i + (k1 + 2*k2 + 2*k3 + k4) / 6.0

    return x_points, y_points

# --- METODE UNTUK BOUNDARY VALUE PROBLEM (BVP) ---

def finite_difference_bvp(beta_sq, x_span, bc, h):
    """Menyelesaikan BVP menggunakan metode Diskritisasi Beda Hingga."""
    x_points = np.arange(x_span[0], x_span[1] + h, h)
    N = len(x_points) - 2 # Jumlah titik interior

    # Membangun matriks A
    A = np.zeros((N, N))
    main_diag = (beta_sq * h**2 - 2) * np.ones(N)
    off_diag = np.ones(N - 1)
    np.fill_diagonal(A, main_diag)
    np.fill_diagonal(A[1:], off_diag)
    np.fill_diagonal(A[:, 1:], off_diag)

    # Membangun vektor b
    b = np.zeros(N)
    b[0] = -bc[0] # dari V_0
    b[-1] = -bc[1] # dari V_N+1

    # Menyelesaikan sistem linear Ay = b
    v_internal = np.linalg.solve(A, b)

    # Menggabungkan hasil dengan kondisi batas
    v_solution = np.concatenate(([bc[0]], v_internal, [bc[1]]))

    return x_points, v_solution

# =====
# 3. EKSEKUSI, ANALISIS, DAN VISUALISASI
# =====
def run_analysis():

```

```

"""Fungsi utama untuk menjalankan seluruh analisis dan membuat plot."""

h_values = [0.1, 0.05, 0.01]
x_span = (x_start, x_end)

# --- Analisis IVP ---
print("--- HASIL ANALISIS INITIAL VALUE PROBLEM (IVP) ---")
ivp_errors = {h: {} for h in h_values}

for h in h_values:
    print(f"\n--- Menganalisis untuk h = {h} ---")

    # Jalankan semua metode IVP
    x_euler, y_euler = euler_method(model_ivp, y0, x_span, h)
    x_mid, y_mid = midpoint_method(model_ivp, y0, x_span, h)
    x_rk4, y_rk4 = rk4_method(model_ivp, y0, x_span, h)

    # Ambil solusi V(x) (komponen pertama dari y)
    v_euler = y_euler[:, 0]
    v_mid = y_mid[:, 0]
    v_rk4 = y_rk4[:, 0]

    # Hitung solusi analitis pada titik yang sama
    v_analytical = analytical_solution(x_euler)

    # Hitung dan simpan error di titik akhir
    error_euler = abs(v_euler[-1] - v_analytical[-1])
    error_mid = abs(v_mid[-1] - v_analytical[-1])
    error_rk4 = abs(v_rk4[-1] - v_analytical[-1])
    ivp_errors[h] = {
        'Euler': error_euler,
        'Midpoint': error_mid,
        'RK4': error_rk4
    }
    print(f"Error di x={x_end:.3f}: Euler={error_euler:.5f},
Midpoint={error_mid:.5f}, RK4={error_rk4:.7f}")

    # Plotting
    plt.figure(figsize=(12, 7))
    plt.plot(x_euler, v_analytical, 'k-', linewidth=2.5, label='Solusi
Analitis')
    plt.plot(x_euler, v_euler, 'r--o', markersize=4, label='Euler')
    plt.plot(x_mid, v_mid, 'g--s', markersize=4, label='Titik Tengah
(Midpoint)')
    plt.plot(x_rk4, v_rk4, 'b--^', markersize=4, label='RK4')
    plt.title(f'Perbandingan Metode IVP untuk h = {h}', fontsize=16)
    plt.xlabel('Posisi (x)', fontsize=12)
    plt.ylabel('Tegangan V(x)', fontsize=12)

```

```

plt.legend(fontsize=12)
plt.grid(True)
plt.show()

# Tampilkan tabel error IVP
print("\n--- Tabel Ringkasan Error Absolut IVP di Titik Akhir ---")
print("h\t\t| Euler \t| Midpoint \t| RK4")
print("-----")
for h, errors in ivp_errors.items():
    print(f"{h:<8}\t| {errors['Euler']:.7f}\t| {errors['Midpoint']:.7f}\t| {errors['RK4']:.9f}")

# --- Analisis BVP ---
print("\n\n--- HASIL ANALISIS BOUNDARY VALUE PROBLEM (BVP) ---")
bvp_errors = {}

for h in h_values:
    print(f"\n--- Menganalisis BVP untuk h = {h} ---")
    x_bvp, v_bvp = finite_difference_bvp(beta**2, x_span, bc, h)
    v_analytical_bvp = analytical_solution(x_bvp)

    # Hitung error maksimum
    max_error = np.max(np.abs(v_bvp - v_analytical_bvp))
    bvp_errors[h] = max_error
    print(f"Error maksimum untuk BVP dengan h={h}: {max_error:.7f}")

# Plotting
plt.figure(figsize=(12, 7))
plt.plot(x_bvp, v_analytical_bvp, 'k-', linewidth=2.5, label='Solusi Analitis')
plt.plot(x_bvp, v_bvp, 'm--o', markersize=5, label='Metode Beda Hingga')
plt.title(f'Solusi BVP Metode Diskritisasi untuk h = {h}', fontsize=16)
plt.xlabel('Posisi (x)', fontsize=12)
plt.ylabel('Tegangan V(x)', fontsize=12)
plt.legend(fontsize=12)
plt.grid(True)
plt.show()

# Tampilkan tabel error BVP
print("\n--- Tabel Ringkasan Error Maksimum BVP ---")
print("h\t\t| Error Maksimum")
print("-----")
for h, error in bvp_errors.items():
    print(f"{h:<8}\t| {error:.7f}")

if __name__ == '__main__':
    run_analysis()

```